UMB: A Unified Markov Binary Format for Probabilistic Model Checking*

Roman Andriushchenko[1,2], Arnd Hartmanns[3], Joshua Jeppson[4], Sebastian Junges[5], Tobias Meggendorfer[6], David Parker[7], Tim Quatmann[8], and Maximilian Weininger[9]

[1] Masaryk University, Brno, Czechia
[2] Brno University of Technology, Brno, Czechia
[3] University of Twente, Enschede, The Netherlands
[4] Utah State University, Logan, Utah, USA
[5] Radboud University, Nijmegen, The Netherlands
[6] Lancaster University Leipzig, Leipzig, Germany
[7] University of Oxford, Oxford, UK
[8] RWTH Aachen University, Aachen, Germany
[9] Ruhr-University Bochum, Bochum, Germany

Abstract. This paper presents the unified Markov binary (UMB) format, an efficient, extensible, and well-supported explicit-state file format for representing a wide range of probabilistic systems. UMB addresses the problem that, while probabilistic model checking tools often support common high-level modelling languages, there is no effective mechanism for exchanging low-level model representations. In practice, textual, tool-specific formats are used, hampering interoperability and resulting in large overheads in writing and reading model files. UMB provides a clean, unified, and efficient solution, based on a general underlying mathematical model, and encoded using a small set of bit-level primitive data structures. The format has already been adopted by prominent tools and comes with a convenient Python library for reading, manipulating, creating, and validating models, plus infrastructure for cross-tool installation and continuous validation. We report on both the efficiency of the file format and the new practical use cases that it facilitates.

1 Introduction

Probabilistic model checking (PMC) is a technique for formal modelling and analysis of stochastic systems [6, 7, 37]. Several mature PMC tools [2, 25, 32, 34, 38, 41] have been developed over the past decades. They support models ranging from Markov chains and Markov decision processes (MDPs) [11, 23, 28] to partially observable MDPs (POMDPs) and stochastic games (SGs) [1]. Since the features and strengths of these tools differ, and because they are commonly incorporated into toolchains, efficient and reliable interconnectivity is crucial.

* The authors are ordered alphabetically. This work was initiated by Dagstuhl Meeting 24134 "Towards A Unified Interface For Modern Probabilistic Model Checking Tools".

As a *high-level* modelling formalism, the textual PRISM language [43] has become a de facto standard for probabilistic modelling and model checking, supported by various tools and covering many types of models. The JSON-based Jani format [10] facilitates the interchange of high-level probabilistic model descriptions between tools. Internally, *low-level* storage and solution of models in probabilistic model checkers is now very often *explicit-state*, i.e., states, transitions and their probabilities are explicitly enumerated in memory. There is a distinct lack of effective *interfaces* for such model representations between tools. In practice, it is usual to either (i) re-encode the models into high-level formats such as the PRISM language, creating a large overhead in storage and parsing, or (ii) use simple, tool-specific textual formats [42, 47], which tend to be ad hoc, limited to certain model types, and also inefficient to generate and parse.

To address this need for a universal low-level model exchange format, we present the *unified Markov binary* (UMB) format, an efficient, extensible, and well-supported file format for the explicit-state representation of probabilistic systems. Working with PMC tool developers and users, we have devised a general-purpose model encoding, along with a clearly defined mapping to the format from the majority of (discrete space, Markovian) model types in common use. This includes Markov chains, MDPs, and their extensions with continuous time, partial observability, uncertain transition probabilities, and multiple players.

The UMB format. We formalise our model encoding with the unifying mathematical representation of *annotated transition systems* (ATSs, Sect. 3) in which transitions between *states* are layered into *choices* and *branches*. These three fundamental objects are then augmented with *annotations*, capturing further model information, such as transition probabilities, atomic propositions, or rewards.

Building on this formalism, we define the UMB file format (Sect. 4), which combines structured metadata and binary files to encode the model data. For the latter, we focus on a small set of bit-level defined primitive data structures, aiming to strike a balance between simplicity and efficiency/compactness. We align with established storage schemes (such as the compressed sparse row, or CSR, format for matrices) to reduce data transformation overheads. Our experiments in Sect. 6 illustrate the resulting gains in processing times.

Moreover, the format has been designed with extensibility in mind. For example, it is straightforward to use combinations of modelling features that still lack tool support, e.g., partially observable stochastic games (POSGs) or interval continuous-time Markov chains (CTMCs). Further, while we *currently* exclude models with *symbolic* expressions (e.g., probabilistic timed automata, which comprise guard expressions over real-valued clock variables [39]), the file format is flexible enough to accommodate these via, e.g., JSON-based annotations.

UMB is supported by at least seven PMC tools, including the widely used Prism [38], mcsta [26], and Storm [32]. It is also accompanied by the umbi Python library, which acts as a reference implementation, offering methods for creating, modifying and analysing models, and the *UMB observatory*, which ensures alignment of UMB-enabled tools through a shared test environment and continuous integration. We describe UMB tooling in Sect. 2 and, in Sect. 5,

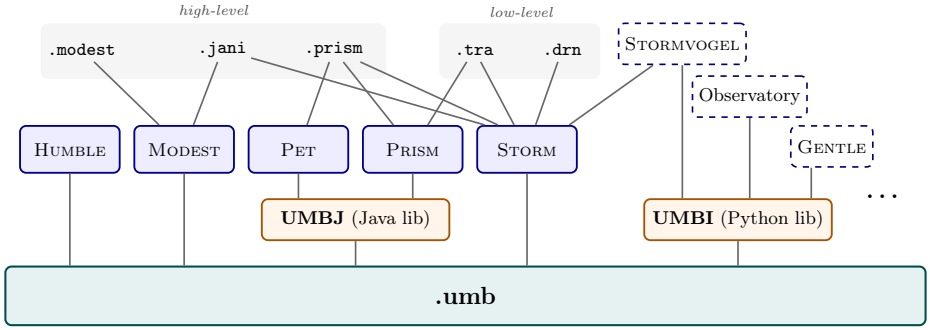


Fig. 1. The UMB ecosystem: Tools supporting UMB, and other existing connections.

highlight a number of novel *use cases* for the format, including toolchains that combine the strengths of different tools and rapid prototyping.

Related file formats. Established modelling formalisms like the PRISM language and Jani serve a different purpose, targeting *high-level* model descriptions at the level of components and variables, defining model behaviour using symbolic expressions. As such, they are poorly suited to encode explicit-state, *low-level* models which do not lend themselves to concise, symbolic representation.

Existing explicit-state file formats for PMC tend to be tool-specific, text-based and with various limitations. Prism uses the `.tra` format [42] to write/read the core transition behaviour, with separate formats for other model entities, such as rewards and atomic propositions. It provides minimal metadata, so the precision of numerical values, for example, must be inferred. Storm offers the `.drn` format [47], which provides a more general, structured textual format, but as a result is more verbose than `.tra`. Both remain somewhat ad hoc formats and are inefficient for reading and writing large models.

Related file formats from other areas include: BCG, a very compact proprietary binary format from the CADP toolset [19], primarily for labelled transition systems (LTSs) but with the facility to add probability labels, albeit in a much more limited fashion than is possible with UMB; and MoXI [35], a model exchange format that focuses on symbolic encodings of non-probabilistic models.

2 The UMB Ecosystem

The ecosystem of UMB-enabled tools, illustrated in Fig. 1, includes many of the PMC tools in common use, as well as a Python library umbi, and the *UMB observatory* for ensuring consistency between tools supporting UMB.

umbi Python Library. umbi, available at github.com/pmc-tools/umbi, is a lightweight Python library and command-line interface for reading, manipulating, creating, and validating UMB files. It makes interacting with the model straightforward by providing low-level file access via concise, well-documented primitives.

The library features a modular and extensible design with small, focused modules and clear extension points for custom adapters or future format extensions.

Beyond file-level operations, UMBI offers a format-agnostic abstraction layer that enables users to manipulate an ATS without understanding UMB internals. The ATS is represented as a simple object model, with entities and their annotations accessible via iterable collections and query helpers, following a first-state-next-state API. By hiding file layout details, this abstraction ensures that code written against the API works regardless of the underlying storage format and remains valid in the face of format changes, streamlining tool integration and allowing a focus on models and algorithms rather than file format specifics.

Example 1. The following snippet shows the use of UMBI to encode a small MDP (given later in Fig. 2, right), and illustrates how to extend it with observations to obtain a POMDP and then with players to produce a POSG.

```
import umbi

# create a 4-state MDP
ats = umbi.ats.SimpleAts()
ats.num_states = 4
ats.initial_states = [0]

ats.new_choice(state=0, action_name="a", targets=[1, 2], probs=[.5, .5])
ats.new_choice(state=0, action_name="b", targets=[2, 3], probs=[.2, .8])
ats.new_choice(state=1, action_name="a", targets=[0, 1], probs=[.9, .1])
ats.new_choice_absorbing(state=2, action_name="tau")
ats.new_choice_absorbing(state=3, action_name="tau")

# add state-to-observation annotation, making it a POMDP
ats.num_observations = 2
ats.observation_annotation.state_values = [0, 0, 1, 1]

# add state-to-player annotation, making it a POSG
ats.num_players = 2
ats.state_to_player = [0, 1, 0, 0]
ats.player_to_name = ["p0", "p1"]

umbi.ats.write(ats, "out.umb")
```

Tool support. UMB is supported by the majority of current PMC tools.

PRISM, as of version 4.10, provides UMB import and export for all models supported by both the tool and the format, notably Markov chains, MDPs and POMDPs, interval variants of all these, and LTSs. This includes rewards, atomic propositions, valuations for states/observations, and either floating-point or exact arithmetic models. PRISM-GAMES, as of version 3.2.2, provides the same functionality for turn-based stochastic games. Support for reading and writing UMB files has been implemented as a standalone `umbj` Java library that is available at github.com/pmc-tools/umbj.

STORM, as of version 1.12, includes UMB support for (either compressed or uncompressed) import and export of Markov automata (MAs), stochastic games, POMDPs, and their special cases, e.g., interval- or rational-valued transitions, and including rich annotations on states and observations. The support is deeply

embedded and thus STORM provides an interface also to, e.g., fault trees or Petri nets, or can be used to provide, e.g., bisimulation quotients of UMB models.

In the MODEST TOOLSET, the MCSTA probabilistic model checker has been rebuilt around the UMB format. Its internal data structures match the files and binary representation of UMB, allowing MCSTA to "load" uncompressed UMB files by simply mapping them into memory via `mmap`. It supports reading UMB files encoding LTSs, discrete-time Markov chains (DTMCs), CTMCs, MDPs, and MAs, and can export such models given in JANI or the MODEST language [9, 24] to UMB. As working with unchecked pointers/array indexing on untrusted UMB data could easily create security vulnerabilities, MCSTA includes a strict UMB validator that thoroughly checks the metadata and binary files.

The tools PAYNT [2], STAMINA [34] and STORMVOGEL [40] use parts of STORM, so also now import and export `.umb` files, the latter connecting via UMBI. Similarly, PET [41] supports UMB through PRISM's `umbj` library. Since the finalisation of UMB in early 2026, at least two new tools built on the format have appeared: The HUMBLE fully verified probabilistic model checker [30], which works directly on UMB files, and the GENTLE DTMC benchmark generator [3] built on UMBI.

UMB observatory. To ensure alignment of UMBI and the various UMB-enabled tools, we provide the *UMB observatory* (pmc-tools.github.io/umb-observatory/), a shared test environment with continuous integration. This is realised as a container with the latest versions of the tools installed, plus thin Python wrappers around the command-line tools to simplify using and configuring them. The observatory includes tests for tools, tool-UMBI compatibility, and cross-tool compatibility. They currently test the exception-free write-read-write-read paths through the tools along with some semantic checks. The tests already demonstrated their benefits during development by detecting many subtle differences between parsers and exporters. Finally, a local Jupyter server with a simple notebook simplifies access to rerun individual tests.

3 Unified Mathematical Representation

Underlying our file format UMB is the unified mathematical representation of *annotated transition systems (ATSs)*. In an ATS, transitions between *states* comprise two layers, *choices* and *branches*; a wide range of *annotations* can be applied to all of these. Many commonly employed models, including MDPs, Markov chains, and a wide range of others supported by probabilistic model checkers can be captured naturally in ATSs. Furthermore, the formalism is designed to map cleanly to a generalisable implementation as a file format.

Definition 1 (ATS). *A (discrete-space, time-homogeneous) ATS is a pair* $\mathcal{S} = (\mathcal{T}, \mathcal{A})$ *of a* transition system $\mathcal{T}$ *and a set of* annotations $\mathcal{A}$*. A transition system is a triple* $\mathcal{T} = (\mathsf{S}, \mathsf{C}, \mathsf{B})$ *consisting of finite sets of* states S*,* choices C*, and* branches B*. Each state* $s \in \mathsf{S}$ *has a (possibly empty) set of choices* $\mathsf{C}_s \subseteq \mathsf{C}$*, and each choice* $c \in \mathsf{C}$ *has a (possibly empty) set of branches* $\mathsf{B}_c \subseteq \mathsf{B}$*. These sets are disjoint, i.e.* $\mathsf{C} = \biguplus_{s \in \mathsf{S}} \mathsf{C}_s$ *and* $\mathsf{B} = \biguplus_{c \in \mathsf{C}} \mathsf{B}_c$*. Further, every branch* $b \in \mathsf{B}$ *defines*

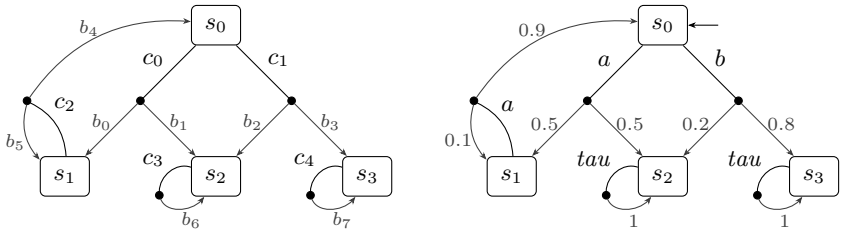


Fig. 2. A transition system (left) and the MDP that results from annotating it with branch probabilities and (choice) actions (right).

a (not necessarily unique) successor state $s_b \in \mathsf{S}$*. An* annotation *is a mapping* $\mathsf{E} \to X$ *from an* entity $\mathsf{E} \in \{\mathsf{S}, \mathsf{C}, \mathsf{B}\}$ *of the transition system to some domain* X*.*

On the left of Fig. 2, we show a transition system with 4 states, 5 choices and 8 branches. We have $\mathsf{C}_{s_0} = \{c_0, c_1\}$, $\mathsf{B}_{c_1} = \{b_2, b_3\}$, and $s_{b_3} = s_3$. Annotations can map to integers $\mathbb{Z}$, reals $\mathbb{R}$, probabilities $[0, 1] \subset \mathbb{R}$, Booleans $\{0, 1\}$, or any other set needed to define the model. Annotations capture both core model data (e.g., transition probabilities or state-to-player mappings in games) and other labellings needed for model checking (e.g., atomic propositions or rewards).

3.1 Representing MDPs

An ATS can capture finite systems with Markovian dynamics, i.e., in which the definition of possible transitions to the next state depends only on the current state. It also naturally supports a combination of both nondeterministic and probabilistic choices. As a first illustration of this, we describe an instantiation of ATSs to MDPs [44], a widely used model in PMC and other fields such as artificial intelligence and control theory. MDPs are the simplest probabilistic model where all components of Definition 1 are non-trivial.

Definition 2. *An MDP is a tuple* (S, s_0, A, P) *where* S *is a finite set of states,* $s_0 \in \mathsf{S}$ *is an initial state,* A *is a finite set of actions, and* $P \colon \mathsf{S} \times A \times \mathsf{S} \to [0, 1]$ *is a transition probability function s.t.* $\forall\, s \in \mathsf{S}, a \in A \colon \sum_{s' \in \mathsf{S}} P(s, a, s') \in \{0, 1\}$*.*

We can naturally encode an MDP as a transition system $\mathcal{T}$ and annotation set $\mathcal{A}$, where the latter includes mappings of the form $\mathsf{S} \to \{0, 1\}$, $\mathsf{C} \to A$, and $\mathsf{B} \to [0, 1]$, representing initial states, actions, and transition probabilities, respectively. To match Definition 2, we can require that a single state is marked as initial and choice actions are distinct within a state, but more general representations could also be permitted. The right of Fig. 2 shows how the example transition system above is annotated to represent a 4-state MDP with three actions $A = \{a, b, tau\}$.

3.2 Property-Related Annotations

MDPs are typically used in conjunction with *rewards* (or, conversely, costs), most frequently assigned to state-action pairs ($\mathsf{S} \times A \to \mathbb{R}$), but also to state-action-state triples ($\mathsf{S} \times A \times \mathsf{S} \to \mathbb{R}$) or states ($\mathsf{S} \to \mathbb{R}$). All three are easily encoded

as annotations for choices, branches and states of an ATS, respectively. MDPs are also often augmented with a labelling function that maps states to a set of *atomic propositions* AP, allowing the specification of temporal logic properties. We capture these with a set of Boolean-valued annotations $\{f_a \colon \mathsf{S} \to \{0,1\} \mid a \in \mathsf{AP}\}$.

Information from a high-level model description can also be attached. A good example is mapping states to *valuations*, i.e., the values of *model variables/factors* (e.g., position, velocity, or protocol state) that make up each state. These can be used in property specification, for debugging/validating model checking results, and for generating concise, understandable policies [4, 16].

3.3 Other Model Classes

ATSs are designed for generality and encode many other common model types:

- *DTMCs* can be represented as MDPs with one choice per state. For *CTMCs*, we adopt the common approach of treating them as a DTMC (the "embedded Markov chain") with an *exit rate* for each state – an annotation $\mathsf{S} \to \mathbb{R}_{\geq 0}$.
- *MAs* [17] generalise CTMCs with nondeterministic choices. They are naturally stored as a combination of the CTMC and MDP encodings, plus a Boolean annotation indicating which states are Markovian.
- *Partially observable* models, such as POMDPs [36], add *observations* O made by an agent as the model evolves. These are typically assigned to action-successor tuples $Z \colon A \times \mathsf{S} \to O$ or to states $Z \colon \mathsf{S} \to O$, which is representable as annotations of branches and states, respectively. *Stochastic* observations are also considered, where Z instead maps to distributions over observations.
- *Interval models* are a common way to capture epistemic uncertainty in probabilistic systems. Interval Markov chains, interval MDPs [20], and even interval POMDPs are directly supported in ATSs by changing the type of the branch probability annotation from $[0,1]$ to $[0,1]^2$.
- *Stochastic games* generalise MDPs by allowing multiple *players* to resolve how choices are taken in each state. For turn-based games [15], we add an annotation mapping states to players, indicating control. For concurrent games [46], an annotation of choices with tuples of players would be used.
- Lastly, although our focus is Markovian models, *non-probabilistic* models are easily supported, too; e.g., an LTS [7] is a special MDP with only one branch per choice and no branch probability annotation.

Table 1 summarises how these model classes fit into ATSs. Each row is a model type, and the columns show which entities and annotations it uses: A ✓ indicates usage, 𝟙 indicates trivial usage with a single choice per state or a single branch per choice, and $\checkmark^2$ marks double usage, for both probabilities and rates. The columns correspond to fields of the `"transition-system"` object in the JSON-encoded metadata of a UMB file, which defines the model type (see Sect. 4 below). Note that the modelling features above are all orthogonal, so we can easily represent "exotic" combinations like partially observable stochastic games or interval MAs, greatly simplifying the process of adding new model variants. This flexibility and extensibility are key benefits of our formalism.

Table 1. Summary of how established model types map to ATSs.

Model type	States	Choices	Branches	#Players	Observ.ns	Branch prob.s	Exit rates
Markov decision process	✓	✓	✓	1	-	✓	-
Discrete-time Markov chain	✓	1	✓	0	-	✓	-
Continuous-time Markov chain	✓	1	✓	0	-	✓	✓
Markov automaton	✓	✓	✓	1	-	✓[2]	✓
Partially observable MDP	✓	✓	✓	1	✓	✓	-
Interval Markov chain	✓	1	✓	0	-	Intervals	-
Interval MDP	✓	✓	✓	1	-	Intervals	-
Turn-based stochastic game	✓	✓	✓	>1	-	✓	-
Labelled transition system	✓	✓	1	1	-	-	-

4 The UMB File Format

The UMB format is the product of detailed discussions among PMC tool developers and users, aiming for a *unified*, widely supported approach to cleanly represent a broad range of probabilistic models. We now provide an overview of its practical details, continuing our illustrative example. The complete technical specification of UMB is available online at:

`https://github.com/pmc-tools/umb/blob/main/spec.md`.

4.1 Data Structures

The transition system $\mathcal{T}$ of an ATS can be represented as two one-to-many mappings, $\mathsf{SC}\colon \mathsf{S} \to 2^{\mathsf{C}}$ of states to choices and $\mathsf{CB}\colon \mathsf{C} \to 2^{\mathsf{B}}$ of choices to branches that partition the respective sets, plus a mapping $\mathsf{BS}\colon \mathsf{B} \to \mathsf{S}$ of branches to successor states. Assume strict total orders on states $<_{\mathsf{S}}$, choices $<_{\mathsf{C}}$, and branches $<_{\mathsf{B}}$ so that $s <_{\mathsf{S}} s' \Rightarrow \forall c \in \mathsf{C}_s, c' \in \mathsf{C}_{s'} : c <_{\mathsf{C}} c'$ and analogously for choices and branches. Let us identify the states, choices, and branches in these orders by zero-based indices up to $n-1$, $m-1$, and $l-1$, respectively.

In UMB, we then represent BS by a vector $\mathbf{BS} = (bs_k)$ of length l where bs_k is the index of the successor state of the branch with index k. For annotations of states, choices, or branches we similarly use a vector of length n, m, or l, respectively, that stores the values each entity is mapped to. For SC and CB, we adopt a representation based on the CSR storage scheme for sparse matrices [45]: For SC, we use a vector $\mathbf{SC} = (sc_i)$ of length $n+1$ with $sc_i = |\bigcup_{i'<i} \mathsf{C}_{i'}|$, storing for each state the index of its first choice plus a final entry of m. Then, for a state with index i, the indices of all its choices are $\{sc_i, \ldots, sc_{i+1} - 1\}$. In the same way, $\mathbf{CB}$ of length $m+1$ implements CB.

Fig. 3 shows how UMB represents the example MDP of Fig. 2 using the three vectors $\mathbf{SC}$, $\mathbf{CB}$, and $\mathbf{BS}$. We include the following annotations: the initial states,

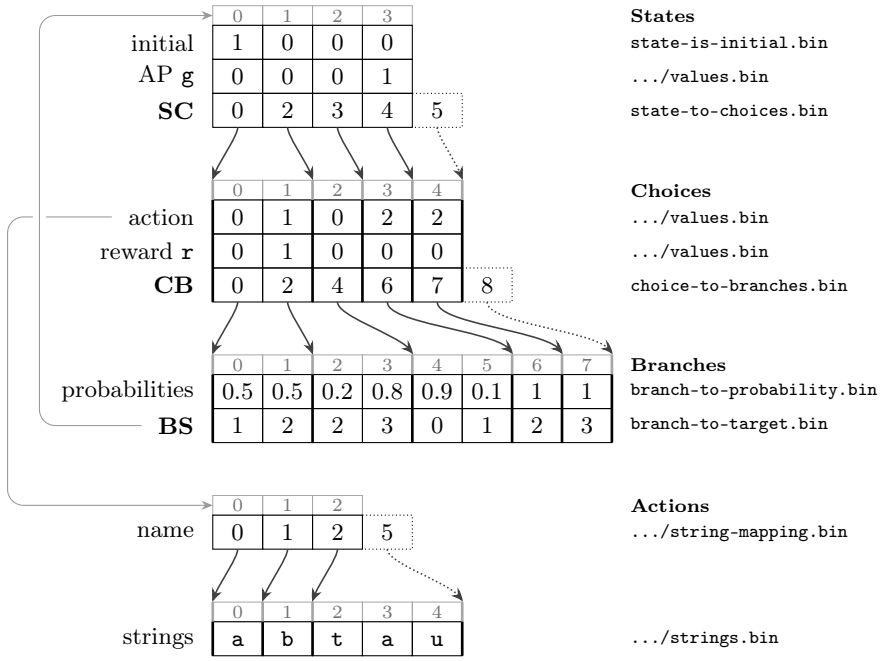


Fig. 3. Visualisation of the UMB representation of the example MDP.

an atomic proposition (AP) `g` (labelling only state s_3), the mapping of choices to actions, a reward structure `r` on choices (that assigns reward 1 to state-action pair (s_0, b)), and the branch probabilities. All annotations map to fixed-size objects except for the action names: they are stored as a sequence of bytes indexed by another CSR vector. In the figure, every block of rows belongs to one kind of entity, with the entities' indices in grey on top; a grey arrow indicates that the values of a row are indices into the block it points to. A black arrow points from an entry of a CSR vector to the start of its range in the block below, where thick lines separate the ranges; the dotted final entries mark the end of the last range.

4.2 File Format

A `.umb` file is a tar file, optionally compressed with GZip or XZ, that contains a single JSON index file `index.json` plus a set of binary `.bin` files.

Metadata and index. The JSON index file contains metadata about the model. In particular, it specifies the type of the model (e.g., whether it is discrete- or continuous-time and single- or multi-player), basic statistics (such as the number of initial states), and what annotations are included. In addition to states, choices, and branches, two ATS-level annotations are first-class UMB entities: the players of game-based models, and the observations of partially observable models. Furthermore, the common annotations for initial and Markovian states, exit rates, actions, probabilities, and state valuations are also first-class concepts.

Core model data. UMB's binary files come in three kinds implementing the three types of vectors seen earlier: `CSR` files storing CSR vectors with unsigned 64-bit indices for one-to-many mappings like SC, `TO1` files storing simple vectors of fixed-size elements for one-to-one mappings like BS or annotations, and `SEQ` files storing sequences of potentially variable-size values like the action names (which are typically indexed by `CSR` files). UMB defines a small set of types and their binary representations, including Booleans, numeric types (integers, doubles, and rationals, of configurable sizes), intervals, and strings, for use in `TO1` and `SEQ` files. In particular, Boolean values are stored as 1 bit per value, so a Boolean `TO1` file is a bit set. Together, these files provide constant-time look-up of, for example, the branches of an ATS indexed by state/choice and annotations with values of variable sizes such as strings. They are close to, or match, the internal in-memory representations already used by several PMC tools.

The binary files defining the relations of entities as well as first-class concepts are found under well-defined names in well-defined locations. For example, CB is in `CSR` file `choice-to-branches.bin`, the branches' probabilities are in `TO1` file `branch-to-probability.bin`, and states are associated with players in `state-to-player.bin`. The JSON index file determines whether certain binary files must or must not be present; e.g., if `#players` $= 0$, then `state-to-player.bin` must not be part of the UMB file. UMB defines default interpretations for most files so that they can be omitted in special cases. For example, `CSR` files default to $i \mapsto \{i\}$ mappings, so `state-to-choices.bin` can be omitted for a DTMC (where every state has a single choice).

Annotations. All other ATS annotations carry a string identifier *id* and are partitioned into *key*ed groups, with predefined groups for atomic propositions (mapping states to Booleans, key `aps`) and rewards (of continuous numeric types, key `rewards`). The files defining each annotation are in subfolder `annotations/`*key*`/`*id*`/`*entity*, consisting of `values.bin` for the `TO1` mapping from entity to annotation and optional additional files for variable-size string annotations or distributions over annotations. Actions, observations, and valuations (see below) follow the same basic setup in their own specific folders.

Valuations. A model can optionally include, for each entity type (not just states), a set of *valuations*: annotations that map from the entities to tuples of values for a set of *variables*. The primary use is to provide a definition of the meaning of states (and, for partially observable models, observations) with respect to a set of variables (or state factors) in a high-level model description. For compactness, each valuation's data is bit-packed, following a configurable storage scheme specified in the metadata. The scheme's translation to a bit representation is precisely defined by the UMB specification.

Example. Our visualisation in Fig. 3 also shows the names of the files inside the `.umb` file on its right. We abbreviate subfolders as `...`; for example, `values.bin` of atomic proposition `g` would be inside `annotations/aps/g/states`. We further extend the underlying example MDP with state valuations for each state using two variables `x` and `b`. Fig. 4 shows a textual representation of the complete `.umb`

```
/index.json: {
  "format-version": 1,
  "format-revision": 0,
  "model-data": {},
  "file-data": {
    "tool": "PRISM",
    "tool-version": "4.11",
    "creation-date": 1791908954
  },
  "transition-system": {
    "time": "discrete",
    "#players": 1,
    "#states": 4,
    "#initial-states": 1,
    "#choices": 5,
    "#choice-actions": 3,
    "#branches": 8,
    "#branch-actions": 0,
    "#observations": 0,
    "branch-probability-type": {
      "type": "double", "size": 64
  } },
  "annotations": {
    "aps": {
      "g": {
        "alias": "g",
        "applies-to": [ "states" ],
        "type": {
          "type": "bool", "size": 1
        }
    } },
    "rewards": {
      "r": {
        "alias": "r",
        "applies-to": [ "choices" ],
        "type": {
          "type": "double", "size": 64
        }
  } } },
  "valuations": {
    "states": {
      "unique": true,
      "classes": [ {
        "variables": [ {
          "name": "x",
          "type": {
            "type": "uint", "size": 2
          }
        }, {
          "name": "b",
          "type": {
            "type": "bool", "size": 1
          }
        }, { "padding": 5} ]
      } ]
} } }
/state-to-choices.bin:
[0,2,3,4,5]
/choice-to-branches.bin:
[0,2,4,6,7,8]
/branch-to-probability.bin:
[0.5,0.5,0.2,0.8,0.9,0.1,1.0,1.0]
/branch-to-target.bin:
[1,2,2,3,0,1,2,3]
/state-is-initial.bin:
[1000]
/actions/choices/string-mapping.bin:
[0,1,2,5]
/actions/choices/strings.bin:
[a,b,tau]
/actions/choices/values.bin:
[0,1,0,2,2]
/annotations/aps/g/states/values.bin:
[0001]
/annotations/rewards/r/choices/values.bin:
[0.0,1.0,0.0,0.0,0.0]
/valuations/states/valuations.bin:
[.....|0|00,.....|0|01,.....|0|10,.....|1|10]
```

Fig. 4. A textual representation of the `.umb` file for the example MDP.

file. We generated it with PRISM 4.11, whose UMB export has a `text` option that writes this representation directly. For the state valuations, we show the bit-packed layout, with dots for padding bits.

4.3 Design Goals

The design of the format has been driven by several overarching goals. One is to support not only the full range of model classes currently in common use, but also a wide range of *extensions* and *generalisations*. The annotation-based approach enables this, as does the way we store model metadata: for example, rather than encoding the specific model type (e.g., MDP, DTMC), we specify each modelling feature (e.g., nondeterminism, observations, players) separately.

Another goal is a good balance between *usability* and *efficiency*: usability applies to both tool developers and tool users, and efficiency covers export/import speed and compactness of storage. For example, the storage schemes for model data, using CSR and vectors of primitive types, allow tools to work directly

on memory-mapped uncompressed UMB files; all values are stored in little-endian byte order, matching the in-memory layout on common platforms, and `index.json` should come first in the tar file so that tools can read a UMB file as a stream. We do not, however, minimise the size of every value: indices of states, choices, and branches always use 64 bits, for example, instead of the smallest sufficient width or a variable-length encoding. This avoids case distinctions in parsers, and compression recovers the space where file size matters (see Sect. 6). The default interpretations of absent binary files in turn avoid storing redundant data. For numeric values, we support double-precision floating-point numbers for efficiency and compatibility with existing tools, and rationals, with numerator and denominator of the same width to keep indexing simple, for exact results.

Finally, we deliberately keep the first version of UMB simple: it does not support models with *symbolic* expressions (e.g., clock guards in real-time models or rational polynomial probabilities in parametric MDPs). At the ATS level, these are just further annotations, but storing them requires an agreed representation of expressions. We may encode them in JSON, like the metadata, instead of defining a custom grammar. The format already supports string annotations, and its version number, which distinguishes compatible revisions from breaking changes, allows adding this without invalidating existing files.

5 Applications for UMB

To illustrate UMB's broad applicability, we now highlight a number of use cases.

Explicit-state model communication. Interfacing with model checkers via high-level description languages like the PRISM language or Jani is widespread in tools and benchmarks, even when explicitly listing states rather than providing high-level descriptions. Examples include random benchmark generation [5], converters from ASCII to Jani [21], Julia to PRISM for POMDPs [33], and many others. We conjecture that this practice is partially due to the prominence of high-level description languages, their relative ease of use, and their tool-independent support, while explicit-state formats like `.tra` and `.drn` remain less well known.

However, encoding a transition system explicitly in high-level languages is highly inefficient: it massively blows up the representation, and yields files that are notoriously slow to parse (see [5] and Sect. 6.4). By contrast, UMB offers an efficient, easy-to-use, and widely supported alternative: users can describe the transition system on an abstract layer (e.g., programmatically in Python through umbi), enabling better integration with other tools, more maintainable and understandable generation scripts (see github.com/pmc-tools/umbi/tree/main/examples for examples), and more effective evaluation pipelines.

New tool interactions. Many PMC tools have a niche where they are best, offering some Pareto optimal point in terms of modelling features, solving time, guarantees provided, and exploiting structural properties of certain models [1]. UMB allows previously impossible combinations of (i) tool frontends and backends, and of (ii) different transformation and analysis algorithms implemented

in different tools. It also facilitates (iii) turning PMC tools into components of larger toolchains and portfolios. We highlight three examples:

(1) MCSTA is the only PMC tool that supports (MODEST and JANI) models with dynamically-sized arrays and user-defined recursive data types. It is thus used for case studies that can only be succinctly or modularly represented using these features, e.g. [48]. At the same time, STORM's model checking algorithms tend to be faster [11]. Using UMB, these two strengths can be combined: Let MCSTA explore the state space and save it to UMB, which STORM then reads and model-checks—all with little overhead due to the efficiency of UMB.

(2) Vice-versa, for MAs, STORM provides the Unif+ algorithm [14], while MCSTA also implements the switch-step algorithm [12], which performs much better on some models [13]. Yet STORM offers an efficient bisimulation minimisation for MAs [31, Chp. 5]. Via UMB, STORM can now generate, minimise, and export the state space of the hard *bitcoin-attack* MA model [18,27], which MCSTA can load and model-check with the switch-step algorithm:

```
storm --jani ba.jani -const MALICIOUS=40,CD=50 --janiproperty \
      -e dd-to-sparse --bisimulation --exportbuild ba.umb
modest mcsta ba.umb ba.properties.txt --time-bounded-alg SwitchStep
```

The STORM command took under 3 seconds to generate the 11475-state state space, minimise it to 7551 states, and write the UMB file on one author's laptop. MCSTA then checked the model's time-bounded reachability property in 13 seconds. Trying to check the same property with STORM using Unif+, we aborted when no result was returned after 2 minutes. The overhead of going through UMB is clearly negligible compared to the model checking time and the time it saves.

(3) Using UMB, we can easily create simple wrappers that call multiple PMC tools on the same state space, explored and exported to UMB only once, in a portfolio approach to improve performance (as is common in, e.g., software verification [8]). Prior to UMB, such an approach would involve one state space exploration—which may take significant time [1, Sec. 11]—in each tool. Similarly, multiple properties can be checked in parallel; for example, a small, partial state graph can be produced by STAMINA and then potentially dozens of properties can be checked in parallel by multiple instances of STORM or PRISM.

Rapid prototyping. As argued in Sects. 3 and 4, UMB is, by design, easily extensible and can express model classes for which currently only prototypical or even no algorithms exist, such as interval variants of classical models [1, Sec. 10]. Now, using UMBI, one can create or parse example models of these classes, and use this interface to develop and compare solution algorithms. Consequently, UMB facilitates pushing the frontiers of PMC. Crucially, researchers working on analysis algorithms can focus on those, processing UMB files using one of the several parsers available, e.g., UMBI, instead of having to design a modelling language and implement a parser and state space exploration engine.

6 Format Efficiency

We now evaluate the efficiency of UMB experimentally, based on three questions:

Q1 How fast can tools load a UMB file, compared to loading other explicit-state formats and to building the state space from a high-level model? (Sect. 6.1)

Q2 How fast can tools write UMB files, and how large are these files, compared to other explicit-state formats? (Sect. 6.2)

Q3 How does loading from UMB affect total model checking time? (Sect. 6.3)

We answer these questions for three tools on a common benchmark set. Finally, at the end of this section, we quantify the cost of the common practice of encoding explicit-state models in a high-level language, which UMB makes obsolete.

Experimental setup. We ran all experiments on Intel Xeon Platinum 8468 systems running 64-bit Rocky Linux 9.8. We limited each run to 4 CPU cores and 32 GiB of memory using the Slurm batch system, with a timeout of 7200 seconds. All files were read from and written to an NVMe SSD. We use the three general-purpose probabilistic model checkers that natively support UMB:

- Prism version `4.11 f118f0b` from prismmodelchecker.org,
- Storm version `1.14.0 2c6a7c2` from stormchecker.org, and
- mcsta version `v3.1.311 g14c460466` from modestchecker.net.

Formats. For each tool, we compare UMB with the other formats it supports. As input, these are the high-level languages Jani [10] and the PRISM language [43], as well as the textual explicit-state formats `.drn` [47] for Storm and `.tra` [42] for Prism. As output, these are `.drn` for Storm, `.tra` for Prism, and the textual formats `.mrm` (from the IMCA format [22]) and `.aut` (from CADP) for mcsta.

A `.umb` file is a tar archive that can optionally be compressed (see Sect. 4). We thus consider uncompressed UMB files as well as GZip- and XZ-compressed ones, marked by the subscripts "gz" and "xz". For Prism and Storm, we compress the `.tra` and `.drn` files in the same way (for Prism, `.tra` is first combined with the encoding of rewards and atomic propositions into a single so-called `.pexp` file [42]). We run Prism with its explicit engine and Storm with its sparse engine, so that all tools build an explicit-state representation. The only exception is the configuration JANI_{dd}, where Storm uses its decision diagram-based engine to build a symbolic representation of the Jani model instead.

To also compare with a binary format, we convert the `.aut` files to CADP's BCG format using `bcg_io` of CADP version 2025-l and compress the BCG files with the `gzip` and `xz` command-line tools with their default settings. The `.aut` format describes an LTS, so mcsta has to encode choices and branches as transitions: For Markov chains, every branch becomes a transition labelled with its probability or rate. For MDPs, every choice becomes an `i`-labelled transition from its state to a fresh intermediate state, from which one probability-labelled transition per branch leads to the branch's target state. The `.aut` files, and thus the BCG files, contain no further information, whereas the UMB files written by mcsta also include the models' rewards and atomic propositions.

Benchmarks. We consider the benchmarks of QComp 2019 and 2020 [11, 23], taken from the Quantitative Verification Benchmark Set [29]. From these, we

select the 36 DTMC, CTMC, and MDP models that come from PRISM language files (so that all three tools can process them). Within our time and memory limits, we then obtain UMB files and measurements for 32 models with Storm, 30 with mcsta, and 29 with Prism. For *egl.10-8*, *pnueli-zuck.10*, and *rabin.10*, all three tools run out of memory. mcsta does not support *bluetooth.1* and *herman.15* due to multiple initial states and crashes on *oscillators* due to its large high-level language description (37 MiB Jani file). Prism additionally runs out of memory on *bluetooth.1*, *egl.10-2*, *pacman.60*, and *pacman.100*. Storm times out on *bluetooth.1*.

Measurements. The *import time* is the time that a tool needs until it has a complete explicit-state representation of the model in memory. For explicit-state formats, this is the time to read and parse the file. For high-level languages, it includes parsing the model and exploring its state space. The *export time* is the time to write an explicit-state file once the state space is in memory, and we measure the size of the resulting file in bytes. The *total time* is the time to obtain the explicit-state representation and to check one property on it. We repeat each experiment 5 times and report the median. When we report the factor between two times, we count times below 0.1 seconds as 0.1 seconds.

The total time is the wallclock time of the complete tool run. For mcsta, we stop after the state space exploration (`--stop-after Exploration`) so that no model checking is involved, and take the import time from the times that it reports. Its timers have a resolution of 0.1 seconds, so it reports 0.0 seconds for 18 of the 30 uncompressed UMB files. For compressed UMB files, its load time includes decompressing and validating the file.

Presentation. Fig. 5 shows our results for Q1 and Q2 as quantile plots, with one row per tool and one line per format. A point (x, y) on a line indicates that, for this format, the x fastest models each take at most y seconds, or the x smallest files each have at most y bytes. A line ends early if the tool did not produce a result for the remaining models. All y-axes are logarithmic.

6.1 Import Time

High-level modelling languages like Jani and PRISM's require a tool to construct an explicit-state representation in memory before it can perform PMC. The left column of Fig. 5 shows that loading a UMB file is significantly faster than this construction for all three tools. It is also significantly faster than loading the textual explicit-state format `.tra` and `.drn`. For Storm, loading an uncompressed UMB file is faster than building the model from Jani by a median factor of 40, from the PRISM language by a median factor of 40, and faster than parsing the `.drn` file by a median factor of 23. Storm loads every UMB file in under 3 seconds. In contrast, it needs up to 700 seconds for Jani models and up to 2680 seconds for `.drn` files. Building a symbolic representation (JANI_{dd}) is faster than building an explicit one for many models, but still slower than loading UMB. The other tools show the same trend: mcsta loads UMB faster than it explores Jani models by a median factor of 38, and Prism loads UMB faster than it builds PRISM language models by a median factor of 17.

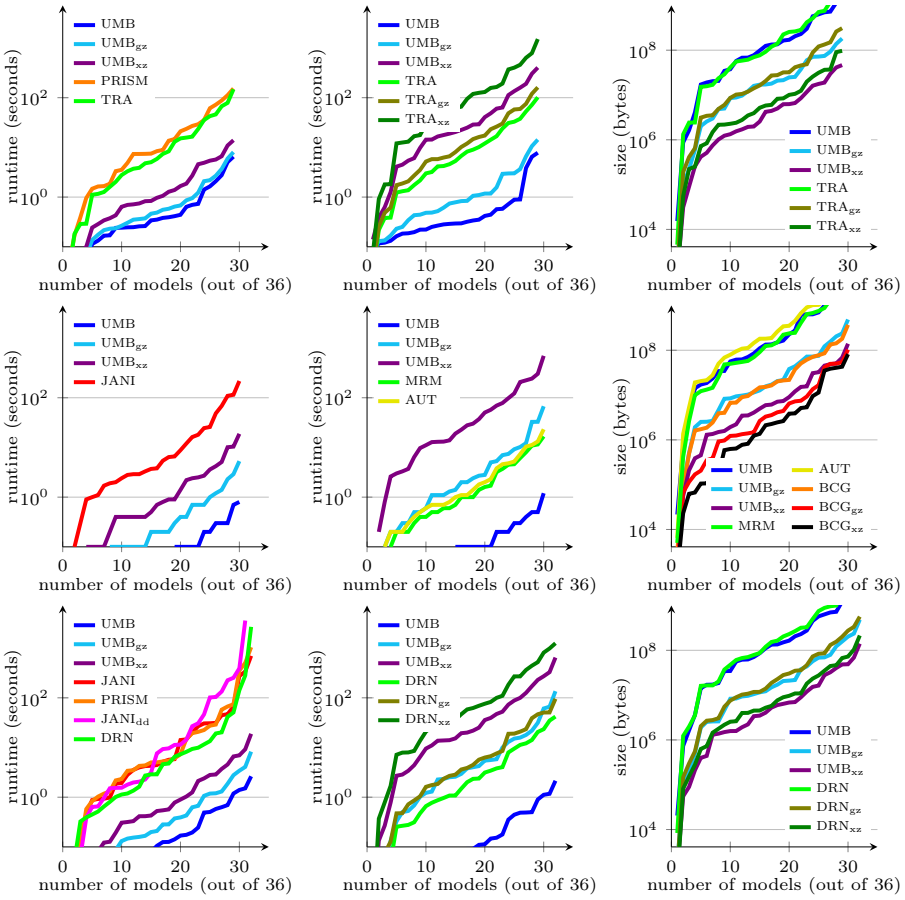


Fig. 5. Format efficiency for PRISM (top), MCSTA (middle), and STORM (bottom). From left to right, we report (i) the import time, (ii) the export time, and (iii) the file size of explicit-state formats, also including compression.

For STORM, GZip-compressed UMB files take a median factor of 1.8 longer to load than uncompressed ones, and XZ-compressed files a median factor of 4.3. Nevertheless, STORM loads even XZ-compressed UMB files at least as fast as the JANI, PRISM language, and uncompressed `.drn` versions of every model, with a single exception: for *speed-ind.2100*, the XZ-compressed UMB file takes 6 % longer than building the model from the PRISM language.

6.2 Export Time and File Size

The middle column of Fig. 5 shows the time to export a model. All tools write uncompressed UMB significantly faster than their textual formats: The median factor is 17 for STORM compared to `.drn`, 16 for PRISM compared to `.tra`, and 8.5 and 10.5 for MCSTA compared to `.mrm` and `.aut`, respectively. As expected,

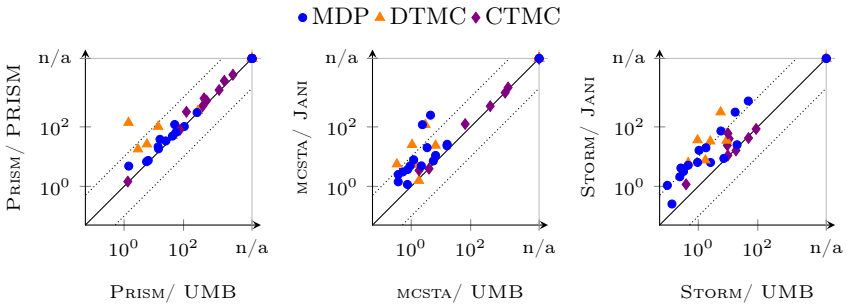


Fig. 6. Total model checking time in seconds of PRISM (left), MCSTA (middle), and STORM (right) when using UMB vs. a high-level language as input.

compression slows down the export of all formats significantly, and then dominates the export time. In STORM, with GZip, the UMB and `.drn` files take about the same time; with XZ, UMB remains faster by a median factor of 2.4.

The right column of Fig. 5 shows the file sizes. Although UMB is a binary format, uncompressed UMB files are of similar size as the textual files: In the median, `.drn` files are 1.2 times and `.aut` files 1.5 times as large as the UMB files, `.tra` files have about the same size, and `.mrm` files are 14 % smaller. This is because UMB stores values at fixed widths, which is what makes UMB files fast to load, whereas the textual formats write small numbers with few characters. Compression removes this redundancy: GZip reduces the size of the UMB files by a median factor of 7, and XZ by a median factor of 23. As usual, XZ thus compresses better than GZip, but is slower. The `.drn` files compress similarly well, but remain larger than the UMB files by a median factor of 1.5 with XZ.

BCG, by contrast, is a binary format specifically designed for compactness. Uncompressed BCG files are smaller than uncompressed UMB files by a median factor of 9. XZ-compressed UMB files, however, are smaller than uncompressed BCG files for 28 of the 30 models. When compressing both formats with XZ, the BCG files are about half the size of the UMB files in the median. BCG is a proprietary format of CADP; other tools can only use it via CADP's `bcg_read` and `bcg_write` libraries, which are subject to CADP's closed license. No probabilistic model checker thus supports it directly, which is why we obtained the BCG files via conversion from `.aut` for this comparison, precluding a fair export time comparison. Based on what we know about BCG, it is not designed for fast random access, unlike uncompressed UMB.

We do not compare the size of UMB files with that of high-level models. High-level models are far smaller than any explicit-state file whenever the state space has a regular structure that the modelling language can express. The median size of the PRISM language files is only 5.2 KiB and that of the JANI files 164 KiB, whereas the UMB files have a median size of 117 MiB. A notable exception is the *oscillators* model, whose PRISM (12 MiB) and JANI (37 MiB) files are larger than its UMB file (1.6 MiB).

6.3 Total Model Checking Time

A faster import only matters in practice if importing is a relevant part of the total time. We thus also measure the entire pipeline. Fig. 6 compares the total times of each tool when given a UMB file (x-axis) vs. a high-level model (y-axis). Each point is one model (MDP, DTMC, or CTMC); for points above the solid diagonal, the UMB input is faster; the dotted lines mark a factor of 10. A point on the *n/a* line means that the tool did not produce a result for the respective input (e.g., timeout or memout). Using UMB as input reduces the total time for almost all models, sometimes by orders of magnitude. For Storm, the median factor is 6.4, the total time at least halves for 23 of the 32 models, and the largest factor is 1400. For mcsta, the median factor is 3.6. The effect is smaller for Prism, with a median factor of 1.3. The reason is the share of the state space construction in the total time: For the high-level inputs, it has a median of 78 % for Storm and 63 % for mcsta, but only 15 % for Prism's explicit engine. On the few models where the UMB input is slower, the difference is at most 17 %.

6.4 Explicit Encodings in High-Level Languages

The experiments above use high-level models that describe their state spaces compactly. As discussed in Sect. 5, however, high-level languages are also commonly used to list the states and transitions of a model explicitly. To evaluate this use case, we generated random transition systems with umbi and encoded them as flat PRISM language files. Prism parses such a file with 10 000 states in a matter of seconds, but cannot parse one with 100 000 states within an hour. This is not a limitation of the tool (as Prism builds models with tens of millions of states from *compact* encodings in a matter of seconds), but a consequence of the explicit encoding, which results in prohibitively large files and requires the tool to process a long, unstructured list of transitions with extensive lookups.

7 Conclusion

We presented the UMB format, a flexible, efficient, and extensible file format for explicit-state probabilistic models, backed by extensive tool support. Our experiments show that UMB files are by far the fastest way to load explicit-state models, and that they are fast to write. They are comparable in size to the textual explicit-state formats, and compress well when size matters. Loading a model from UMB instead of building it from a high-level description also substantially reduces the total model checking time in most cases. Possible future extensions include support for models with symbolic expressions, storage of policies/strategies, and property specification.

Data availability statement. Our Git repositories at github.com/pmc-tools provide access to the software and specifications discussed in this paper; in particular, pmc-tools.github.io/umb-observatory/ gives a hands-on overview. Full benchmark results are available at pmc-tools.github.io/umb-benchmarks. An artifact to reproduce our experiments will be submitted for evaluation.

References

1. Andriushchenko, R., Bork, A., Budde, C. E., Češka, M., Hahn, E. M., Hartmanns, A., Israelsen, B., Jansen, N., Jeppson, J., Junges, S., Köhl, M. A., Könighofer, B., Křetínský, J., Meggendorfer, T., Parker, D., Pranger, S., Quatmann, T., Ruijters, E., Taylor, L., Volk, M., Weininger, M., Zhang, Z.: Tools at the frontiers of quantitative verification: QComp 2023 competition report. In: Beyer, D., Hartmanns, A., Kordon, F. (eds.) TOOLympics Challenge 2023 – Updates, Results, Successes of the Formal-Methods Competitions (TOOLympics 2023), pp. 90–146. Springer (2024). https://doi.org/10.1007/978-3-031-67695-6_4
2. Andriushchenko, R., Ceska, M., Junges, S., Katoen, J.-P., Stupinský, S.: PAYNT: A tool for inductive synthesis of probabilistic programs. In: Silva, A., Leino, K. R. M. (eds.) 33rd International Conference on Computer Aided Verification (CAV 2021). Lecture Notes in Computer Science, vol. 12759, pp. 856–869. Springer (2021). https://doi.org/10.1007/978-3-030-81685-8_40
3. Anonymous Authors: Markov chains made to measure, submitted for double-blind review to an ETAPS 2027 conference.
4. Ashok, P., Jackermeier, M., Kretínský, J., Weinhuber, C., Weininger, M., Yadav, M.: dtControl 2.0: Explainable strategy representation via decision tree learning steered by experts. In: Groote, J. F., Larsen, K. G. (eds.) 27th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2021). Lecture Notes in Computer Science, vol. 12652, pp. 326–345. Springer (2021). https://doi.org/10.1007/978-3-030-72013-1_17
5. Azeem, M., Evangelidis, A., Kretínský, J., Slivinskiy, A., Weininger, M.: Optimistic and topological value iteration for simple stochastic games. In: Bouajjani, A., Holík, L., Wu, Z. (eds.) 20th International Symposium on Automated Technology for Verification and Analysis (ATVA 2022). Lecture Notes in Computer Science, vol. 13505, pp. 285–302. Springer (2022). https://doi.org/10.1007/978-3-031-19992-9_18
6. Baier, C., de Alfaro, L., Forejt, V., Kwiatkowska, M.: Model checking probabilistic systems. In: Clarke, E. M., Henzinger, T. A., Veith, H., Bloem, R. (eds.) Handbook of Model Checking, pp. 963–999. Springer (2018). https://doi.org/10.1007/978-3-319-10575-8_28
7. Baier, C., Katoen, J.-P.: Principles of model checking. MIT Press (2008)
8. Beyer, D., Kanav, S., Kleinert, T., Richter, C.: Construction of verifier combinations from off-the-shelf components. Formal Methods Syst. Des. **66**(1), 99–130 (2025). https://doi.org/10.1007/S10703-024-00449-Y
9. Bohnenkamp, H. C., D'Argenio, P. R., Hermanns, H., Katoen, J.-P.: MoDeST: A compositional modeling formalism for hard and softly timed systems. IEEE Trans. Software Eng. **32**(10), 812–830 (2006). https://doi.org/10.1109/TSE.2006.104
10. Budde, C. E., Dehnert, C., Hahn, E. M., Hartmanns, A., Junges, S., Turrini, A.: JANI: quantitative model and tool interaction. In: Legay, A., Margaria, T. (eds.) 23rd International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2017). Lecture Notes in Computer Science, vol. 10206, pp. 151–168 (2017). https://doi.org/10.1007/978-3-662-54580-5_9
11. Budde, C. E., Hartmanns, A., Klauck, M., Kretínský, J., Parker, D., Quatmann, T., Turrini, A., Zhang, Z.: On correctness, precision, and performance in quantitative verification - QComp 2020 competition report. In: Margaria, T., Steffen, B. (eds.) 9th International Symposium on Leveraging Applications of Formal Methods, Verification and Validation (ISoLA 2020). Lecture Notes in Computer Science, vol. 12479, pp. 216–241. Springer (2020). https://doi.org/10.1007/978-3-030-83723-5_15

12. Butkova, Y., Fox, G.: Optimal time-bounded reachability analysis for concurrent systems. In: Vojnar, T., Zhang, L. (eds.) 25th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2019). Lecture Notes in Computer Science, vol. 11428, pp. 191–208. Springer (2019). https://doi.org/10.1007/978-3-030-17465-1_11
13. Butkova, Y., Hartmanns, A., Hermanns, H.: A Modest approach to Markov automata. ACM Trans. Model. Comput. Simul. **31**(3), 14:1–14:34 (2021). https://doi.org/10.1145/3449355
14. Butkova, Y., Hatefi, H., Hermanns, H., Krcál, J.: Optimal continuous time Markov decisions. In: Finkbeiner, B., Pu, G., Zhang, L. (eds.) 13th International Symposium on Automated Technology for Verification and Analysis (ATVA 2015). Lecture Notes in Computer Science, vol. 9364, pp. 166–182. Springer (2015). https://doi.org/10.1007/978-3-319-24953-7_12
15. Condon, A.: The complexity of stochastic games. Inf. Comput. **96**(2), 203–224 (1992). https://doi.org/10.1016/0890-5401(92)90048-K
16. David, A., Jensen, P. G., Larsen, K. G., Mikucionis, M., Taankvist, J. H.: Uppaal Stratego. In: Baier, C., Tinelli, C. (eds.) 21st International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2015). Lecture Notes in Computer Science, vol. 9035, pp. 206–211. Springer (2015). https://doi.org/10.1007/978-3-662-46681-0_16
17. Eisentraut, C., Hermanns, H., Zhang, L.: On probabilistic automata in continuous time. In: 25th Annual IEEE Symposium on Logic in Computer Science (LICS 2010). pp. 342–351. IEEE Computer Society (2010). https://doi.org/10.1109/LICS.2010.41
18. Fehnker, A., Chaudhary, K.: Twenty percent and a few days – optimising a Bitcoin majority attack. In: Dutle, A., Muñoz, C. A., Narkawicz, A. (eds.) 10th International NASA Formal Methods Symposium (NFM 2018). Lecture Notes in Computer Science, vol. 10811, pp. 157–163. Springer (2018). https://doi.org/10.1007/978-3-319-77935-5_11
19. Garavel, H., Lang, F., Mateescu, R., Serwe, W.: CADP 2011: a toolbox for the construction and analysis of distributed processes. Int. J. Softw. Tools Technol. Transf. **15**(2), 89–107 (2013). https://doi.org/10.1007/S10009-012-0244-Z
20. Givan, R., Leach, S. M., Dean, T. L.: Bounded-parameter Markov decision processes. Artif. Intell. **122**(1-2), 71–109 (2000). https://doi.org/10.1016/S0004-3702(00)00047-3
21. Gros, T. P., Hermanns, H., Hoffmann, J., Klauck, M., Steinmetz, M.: Deep statistical model checking. In: Gotsman, A., Sokolova, A. (eds.) 40th IFIP WG 6.1 International Conference on Formal Techniques for Distributed Objects, Components, and Systems (FORTE 2020). Lecture Notes in Computer Science, vol. 12136, pp. 96–114. Springer (2020). https://doi.org/10.1007/978-3-030-50086-3_6
22. Guck, D., Han, T., Katoen, J.-P., Neuhäußer, M. R.: Quantitative timed analysis of interactive Markov chains. In: Goodloe, A., Person, S. (eds.) 4th International NASA Formal Methods Symposium (NFM 2012). Lecture Notes in Computer Science, vol. 7226, pp. 8–23. Springer (2012). https://doi.org/10.1007/978-3-642-28891-3_4
23. Hahn, E. M., Hartmanns, A., Hensel, C., Klauck, M., Klein, J., Kretínský, J., Parker, D., Quatmann, T., Ruijters, E., Steinmetz, M.: The 2019 comparison of tools for the analysis of quantitative formal models - (QComp 2019 competition report). In: Vojnar, T., Zhang, L. (eds.) 25th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2019).

Lecture Notes in Computer Science, vol. 11429, pp. 69–92. Springer (2019). https://doi.org/10.1007/978-3-030-17502-3_5
24. Hahn, E. M., Hartmanns, A., Hermanns, H., Katoen, J.-P.: A compositional modelling and analysis framework for stochastic hybrid systems. Formal Methods Syst. Des. **43**(2), 191–232 (2013). https://doi.org/10.1007/S10703-012-0167-Z
25. Hartmanns, A., Hermanns, H.: The Modest Toolset: An integrated environment for quantitative modelling and verification. In: Ábrahám, E., Havelund, K. (eds.) 20th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2014). LNCS, vol. 8413, pp. 593–598. Springer (2014). https://doi.org/10.1007/978-3-642-54862-8_51
26. Hartmanns, A., Hermanns, H.: Explicit model checking of very large MDP using partitioning and secondary storage. In: Finkbeiner, B., Pu, G., Zhang, L. (eds.) 13th International Symposium on Automated Technology for Verification and Analysis (ATVA 2015). Lecture Notes in Computer Science, vol. 9364, pp. 131–147. Springer (2015). https://doi.org/10.1007/978-3-319-24953-7_10
27. Hartmanns, A., Hermanns, H.: A Modest Markov automata tutorial. In: Krötzsch, M., Stepanova, D. (eds.) Reasoning Web – 15th International Summer School 2019, Tutorial Lectures. Lecture Notes in Computer Science, vol. 11810, pp. 250–276. Springer (2019). https://doi.org/10.1007/978-3-030-31423-1_8
28. Hartmanns, A., Junges, S., Quatmann, T., Weininger, M.: The revised practitioner's guide to MDP model checking algorithms. Int. J. Softw. Tools Technol. Transf. (2026). https://doi.org/10.1007/s10009-026-00848-y
29. Hartmanns, A., Klauck, M., Parker, D., Quatmann, T., Ruijters, E.: The quantitative verification benchmark set. In: Vojnar, T., Zhang, L. (eds.) 25th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2019). Lecture Notes in Computer Science, vol. 11427, pp. 344–350. Springer (2019). https://doi.org/10.1007/978-3-030-17462-0_20
30. Hartmanns, A., Kohlen, B., Lammich, P.: A Humble approach to verified probabilistic model checking, submitted as a single-blind tool paper to TACAS 2027.
31. Hensel, C.: The probabilistic model checker Storm: symbolic methods for probabilistic model checking. Ph.D. thesis, RWTH Aachen University, Germany (2018), http://publications.rwth-aachen.de/record/752011
32. Hensel, C., Junges, S., Katoen, J.-P., Quatmann, T., Volk, M.: The probabilistic model checker Storm. Int. J. Softw. Tools Technol. Transf. **24**(4), 589–610 (2022). https://doi.org/10.1007/S10009-021-00633-Z
33. Ho, Q. H., Feather, M. S., Rossi, F., Sunberg, Z., Lahijanian, M.: Sound heuristic search value iteration for undiscounted POMDPs with reachability objectives. In: Kiyavash, N., Mooij, J. M. (eds.) 40th Conference on Uncertainty in Artificial Intelligence (UAI 2024). Proceedings of Machine Learning Research, vol. 244, pp. 1681–1697. PMLR (2024), https://proceedings.mlr.press/v244/ho24b.html
34. Jeppson, J., Volk, M., Israelsen, B., Roberts, R., Williams, A., Buecherl, L., Myers, C. J., Zheng, H., Winstead, C., Zhang, Z.: STAMINA in C++: modernizing an infinite-state probabilistic model checker. In: Jansen, N., Tribastone, M. (eds.) 20th International Conference on Quantitative Evaluation of Systems (QEST 2023). Lecture Notes in Computer Science, vol. 14287, pp. 101–109. Springer (2023). https://doi.org/10.1007/978-3-031-43835-6_7
35. Johannsen, C., Nukala, K., Dureja, R., Irfan, A., Shankar, N., Tinelli, C., Vardi, M. Y., Rozier, K. Y.: The MoXI model exchange tool suite. In: Gurfinkel, A., Ganesh, V. (eds.) 36th International Conference on Computer Aided Verification (CAV 2024). Lecture Notes in Computer Science, vol. 14681, pp. 203–218. Springer (2024). https://doi.org/10.1007/978-3-031-65627-9_10

36. Kaelbling, L. P., Littman, M. L., Cassandra, A. R.: Planning and acting in partially observable stochastic domains. Artif. Intell. **101**(1-2), 99–134 (1998). https://doi.org/10.1016/S0004-3702(98)00023-X
37. Kwiatkowska, M., Norman, G., Parker, D.: Probabilistic model checking: Applications and trends. In: Bertrand, N., Dubslaff, C., Klüppelholz, S. (eds.) Principles of Formal Quantitative Analysis. Lecture Notes in Computer Science, vol. 15760, pp. 158–173. Springer (2025). https://doi.org/10.1007/978-3-031-97439-7_7
38. Kwiatkowska, M. Z., Norman, G., Parker, D.: PRISM 4.0: Verification of probabilistic real-time systems. In: Gopalakrishnan, G., Qadeer, S. (eds.) 23rd International Conference on Computer Aided Verification (CAV 2011). Lecture Notes in Computer Science, vol. 6806, pp. 585–591. Springer (2011). https://doi.org/10.1007/978-3-642-22110-1_47
39. Kwiatkowska, M. Z., Norman, G., Segala, R., Sproston, J.: Automatic verification of real-time systems with discrete probability distributions. Theor. Comput. Sci. **282**(1), 101–150 (2002). https://doi.org/10.1016/S0304-3975(01)00046-9
40. Leerkes, P., Melse, I., Heck, L., Volk, M., Junges, S.: Stormvogel: Probabilistic model checking for almost everyone. Tech. rep., Radboud University Nijmegen (2025), https://hdl.handle.net/2066/325553
41. Meggendorfer, T., Weininger, M.: Playing games with your PET: extending the partial exploration tool to stochastic games. In: Gurfinkel, A., Ganesh, V. (eds.) 36th International Conference on Computer Aided Verification (CAV 2024). Lecture Notes in Computer Science, vol. 14683, pp. 359–372. Springer (2024). https://doi.org/10.1007/978-3-031-65633-0_16
42. PRISM developers: PRISM explicit model format specification, https://www.prismmodelchecker.org/manual/Appendices/ExplicitModelFiles
43. PRISM developers: PRISM modelling language documentation, https://www.prismmodelchecker.org/manual/ThePRISMLanguage
44. Puterman, M. L.: Markov Decision Processes: Discrete Stochastic Dynamic Programming. Wiley Series in Probability and Statistics, Wiley (1994). https://doi.org/10.1002/9780470316887
45. Saad, Y.: Iterative Methods for Sparse Linear Systems. SIAM (2003)
46. Shapley, L. S.: Stochastic games. Proceedings of the National Academy of Sciences **39**(10), 1095–1100 (1953). https://doi.org/10.1073/pnas.39.10.1095
47. Storm developers: DRN format description, https://www.stormchecker.org/documentation/background/drn.html
48. Waddoups, N., Boe, J., Hartmanns, A., Basu, P., Roy, S., Chakraborty, K., Zhang, Z.: Probabilistic verification for modular network-on-chip systems. In: 27th International Conference on Verification, Model Checking, and Abstract Interpretation (VMCAI 2026). Lecture Notes in Computer Science, Springer (2026). https://doi.org/10.1007/978-3-032-15700-3_18